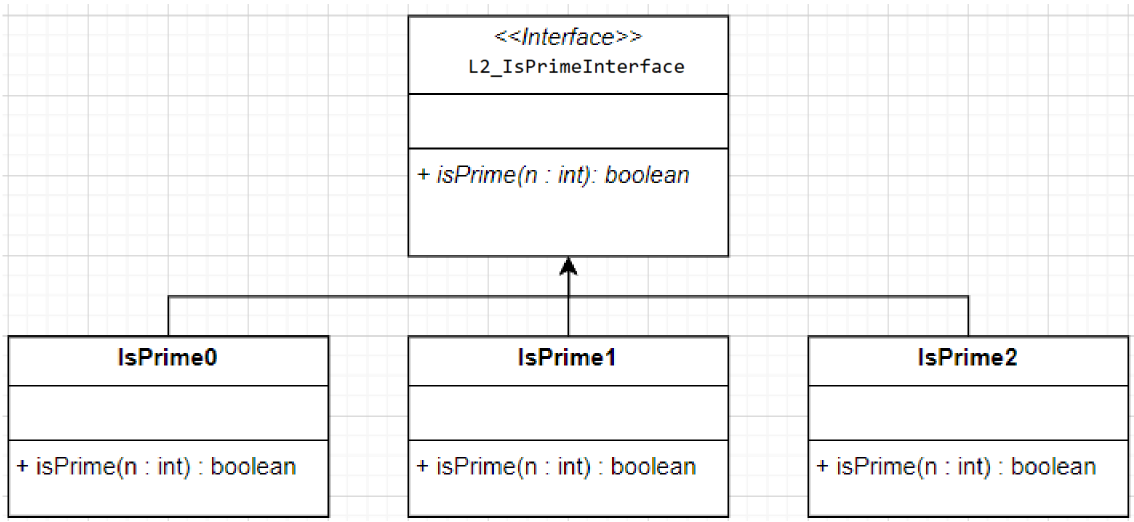
Objective(s):

- To practice on analyzing algorithms' runtime

Task 1: Implement IsPrime0, IsPrime1 and IsPrime2. (in .\Lab02\pack)




```java
package pack;

public interface L2_IsPrimeInterface {
    boolean isPrime(int n);
}
```

```java
// IsPrime2
public boolean isPrime(int n) {
  if (n == 1) return false;
  if (n <= 3) return true;
  if ((n%2 == 0) || (n%3 == 0))
     return false;
  int m = (int)Math.sqrt(n);
  for (int i = 5; i <= m; i += 6) {
      if (n % i == 0) return false;
      if (n % (i+2) == 0) return false;
  }
  return true;
}
```

```java
// IsPrime0
public boolean isPrime(int n) {
  if (n == 1) return false;
  if (n <= 3) return true;
  int m = n/2;
  for (int i = 2; i <= m; i++) {
    if (n % i == 0) return false;
  }
  return true;
}
```

```java
// IsPrime1
public boolean isPrime(int n) {
  if (n == 1) return false;
  if (n <= 3) return true;
  int m = (int)Math.sqrt(n);
  for (int i = 2; i <= m; i++) {
      if (n % i == 0) return false;
  }
  return true;
}
```

The method isPrime0(n) takes any positive integer and returns true if it is prime, and false otherwise. It tests all integers from 2 to n/2 and checks whether any of them divides n.

There are two more methods, isPrime1(n) and isPrime2(n). The method isPrime1(n) is similar to isPrime0(n), but it only runs from 2 to $\sqrt{n}$. The method isPrime2(n) improves on isPrime1(n) by skipping values divisible by 2 or 3 and testing only possible divisors of the form 6k +/- 1.

For testing, we can use the following program:

```
private static void testIsPrime012() {
        int N = 100;
        int count = 0;
        L2_IsPrimeInterface obj = new IsPrime0();
        for (int n = 1; n < N; n++) {
            if (obj.isPrime(n)) count++;
        }
        System.out.println("Pi ("+ N + ")= " + count);
        count = 0;
        obj = new IsPrime1();
        for (int n = 1; n < N; n++) {
            if (obj.isPrime(n)) count++;
        }
        System.out.println("Pi ("+ N + ")= " + count);
        count = 0;
        obj = new IsPrime2();
        for (int n = 1; n < N; n++) {
            if (obj.isPrime(n)) count++;
        }
        System.out.println("Pi ("+ N + ")= " + count);
}
```

Remark : There are 25 prime numbers between 2 to 100. Check the correctness of your implementation.

Task 2: run the program with isPrime0, isPrime1, and isPrime2. Record your result into the following table.

Running-time table					
n	numPrime(n)	time (milliseconds)			
		Lab's isPrime0	isPrime0	isPrime1	isPrime2
100,000		353			
200,000		1,283			
300,000		2,792			
400,000		4,820			
500,000		7,370			
600,000		15,580			
700,000		24,557			
800,000		31,716			
900,000		39,964			
1,000,000		48,785			

```
// edit the driver code to invoke IsPrime0 for bench_isPrime()
public static void bench_isPrime(L2_IsPrimeInterface obj) {
    int your_cpu_factor = 1; /* increase by 10 times */
    int N = 100;
    int count = 0;
    for (N = 100_000; N <= 1_000_000 * your_cpu_factor; N+= 100_000 * your_cpu_factor) {
        count = 0;
        long start = System.currentTimeMillis();
        for (int n = 1; n < N; n++) {
            if (obj.isPrime(n)) count++;
        }
        long time = (System.currentTimeMillis() - start);
        System.out.println(N + "\t" + count + "\t" + time);
        // System.out.printf("%s\t %s\t %s",String.format("%,d",N),
String.format("%,d",count), String.format("%,d",time));
    }
}
```

Task 3 : Analyze whether the running time of isPrime0 grows linearly.

When running algorithms on different input sizes, it is important to understand how the runtime grows. This helps us compare the observed growth rate with the expected Big O complexity. Use these formulas to complete the table below: Time ratio = time(current n) / time(100,000); Increased factor = current time ratio / previous time ratio.

Running-Time Analysis							
n	Data size ratio	Lab's (example) isPrime0	Time Ratio (compared to n)	(Time) Increased Factor	your isPrime0	Time Ratio (compared to n)	(Time) Increased Factor
100,000	n	353	1.0000	-			
200,000	2n	1,283	3.6345	3.6345			
300,000	3n	2,792	7.9095				
400,000	4n	4,820	13.6543				
500,000	5n	7,370	20.4413				
600,000	6n	15,580	44.1359				
700,000	7n	24,557	69.5665				
800,000	8n	31,716	89.8470				
900,000	9n	39,964	113.2124				
1,000,000	10n	48,785	138.2011				

What Should We Expect?

1. Why should the increased factor stabilize?

  - For small n, fixed overhead and timing noise can distort the measurement.

  - When n becomes large enough, the algorithm work dominates the overhead, so the increased factor should become more stable.

2. How should we read the increased factor?

- It compares the current row with the previous row, so it helps show whether the growth pattern is settling.

- A stable increased factor does not prove the exact Big O by itself, but it gives useful evidence about the trend.

3. Does Machine Matter?

If we run the same algorithm on different machines (e.g., Windows vs macOS):

- Raw runtimes will differ due to hardware/OS differences.

- But the time ratios and increase factors should follow a similar pattern.

Time ratios should be similar, even if absolute values change.

Key Takeaways

- Time ratio and increased factor help us reason about algorithm growth.

- Larger input sizes reduce the effect of fixed overhead and timing noise.

- Stable patterns are more meaningful than raw runtimes when comparing different machines.

Task 4: Plot two runtime graphs: your isPrime0 vs. your isPrime1, and your isPrime1 vs. your isPrime2.

Replace the sample values with the times from your table.

```
import matplotlib.pyplot as plt

# Sample input sizes
input_sizes = [100_000, 200_000, 300_000, 400_000, 500_000, 600_000, 700_000,
800_000, 900_000, 1_000_000]

# Sample runtimes in milliseconds (replace with your measurements)
is_prime0 = [100, 200, 300, 400, 500, 600, 700, 800, 900, 1000]          #
isPrime0
is_prime1 = [130, 290, 470, 680, 920, 1190, 1490, 1810, 2150, 2500]       #
isPrime1
is_prime2 = [90, 350, 780, 1400, 2200, 3200, 4400, 5800, 7400, 9200]      #
isPrime2

# Plotting
plt.figure(figsize=(10, 6))
plt.plot(input_sizes, is_prime0, marker='o', label='isPrime0')
plt.plot(input_sizes, is_prime1, marker='s', label='isPrime1')
plt.plot(input_sizes, is_prime2, marker='^', label='isPrime2')

plt.title('Runtime Comparison of isPrime Implementations')
plt.xlabel('Input Size (n)')
plt.ylabel('Runtime (ms)')
plt.grid(True)
plt.legend()
plt.tight_layout()
# export to PNG/PDF using plt.savefig('filename.png')
plt.show()
```

Submission: this pdf. IsPrime0.java IsPrime1.java and IsPrime2.java

Due Date: TBA